

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Engineering Project Report - PCCM</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      line-height: 1.6;
      margin: 40px;
      color: #333;
    }
    h1, h2 {
      color: #1a252f;
      border-bottom: 2px solid #34495e;
      padding-bottom: 5px;
    }
    table {
      width: 100%;
      border-collapse: collapse;
      margin: 20px 0;
    }
    th, td {
      border: 1px solid #ddd;
      padding: 10px;
      text-align: left;
    }
    th {
      background-color: #f2f2f2;
    }
    pre {
      background-color: #f8f9fa;
      border: 1px solid #e9ecef;
      padding: 15px;
      border-radius: 5px;
      overflow-x: auto;
      font-family: Consolas, monospace;
    }
  </style>
</head>
<body>

```

```

<h1>ENGINEERING PROJECT REPORT</h1>

```

```

<p><strong>Project Title:</strong> Pappadam Crispiness Compliance Monitor (PCCM)<br>
<strong>Event:</strong> TinkerHub Useless Projects Hackathon<br>
<strong>Institution:</strong> Saintgits College of Engineering<br>
<strong>Department:</strong> Electronics and Communication Engineering</p>

```

<hr>

<h2>1. Executive Summary</h2>

<p>The Pappadam Crispiness Compliance Monitor (PCCM) is an embedded acoustic testing system designed to evaluate the crispiness of South Asian pappadams. The project addresses the subjective nature of manual crispiness assessment by replacing human sensory evaluation with digital measurement. By measuring acoustic voltage spikes during a controlled fracture test, the device calculates high-frequency sound intensity and displays a binary evaluation ("GRADE A: CRISPY!" or "DEFECT: SOGGY!!!") on an I2C LCD screen.</p>

<h2>2. System Architecture & Hardware Specs</h2>

<p>The system is built around an ATmega328P microcontroller running on a 5V DC supply via USB tethering. Acoustic sampling is performed using an analog sound detection module configured with an electret condenser microphone.</p>

<h3>Key Components</h3>

- Microcontroller: Arduino Nano V3.0 (ATmega328P)Acoustic Sensor: KY-038 Sound Sensor Module (Analog Output Mode)Display Unit: 16x2 Character LCD with PCF8574 I2C Backpack InterfaceControl Input: Tactile Push Button Switch (Internal <code>INPUT_PULLUP</code> Configuration)Prototyping Platform: 400-Point Solderless Breadboard with Solid Core Jumpers

<h2>3. Hardware Interfacing & Pin Mapping</h2>

<thead><tr><th>Component Sub-system</th><th>Terminal Pin Interface</th><th>Arduino Nano Pin</th><th>Signal / Protocol</th></tr></thead><tbody><tr><td>KY-038 Sound Sensor</td><td>Analog Output (AO)</td><td>Pin A0</td><td>Analog Voltage Sampling</td></tr><tr><td>16x2 LCD Display</td></tr></tbody></table>

SDA (Serial Data)	Pin A4	I2C Protocol Data Line
16x2 LCD Display	SCL (Serial Clock)	Pin A5
		I2C Protocol Clock Line
Tactile Push Switch	Terminal 1	Pin D2
		Digital External Interrupt / Trigger
System Power	VCC / GND	5V / GND
		Power Rail (5V DC)

4. Software Implementation & Firmware Design

The system software operates on a non-blocking state machine architecture composed of three primary operational states: `IDLE`, `TESTING`, and `RESULT`.

State Machine Logic

IDLE State: The MCU continuously monitors Digital Pin 2 for an active-low input signal triggered by the user button. The LCD prompts the user to place the sample near the sensor.

TESTING State: Upon activation, a 2000 ms sampling window opens. High-speed continuous analog reads sample the acoustic signal. The peak absolute variance from ambient baseline noise is calculated:

<code>SoundIntensity = |CurrentVolume - AmbientBaseline|</code>

RESULT State: The peak variance (<code>MaxVolumeRead</code>) is evaluated against a predefined threshold (<code>CRUNCH_THRESHOLD = 400</code>). The binary verdict is displayed on the LCD.

5. Firmware Source Code

```
<code>#include <Wire.h>
#include <LiquidCrystal_I2C.h>
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2);

const int SOUND_PIN = A0;
const int BUTTON_PIN = 2;

const int THRESHOLD = 7;

void setup() {
  lcd.init();
  lcd.backlight();

  pinMode(BUTTON_PIN, INPUT_PULLUP);

  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("PAPPADAM TEST");
  lcd.setCursor(0, 1);
  lcd.print("Press START");
}

void loop() {

  // WAIT FOR START BUTTON
  if (digitalRead(BUTTON_PIN) == LOW) {

    delay(50); // debounce

    // START TEST
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("TEST STARTED");
    lcd.setCursor(0, 1);
    lcd.print("Make a CRUNCH!");

    delay(1000);

    // Record background sound
    int baseline = analogRead(SOUND_PIN);

    int maxChange = 0;

    // Listen for 3 seconds
    unsigned long startTime = millis();

    while (millis() - startTime < 3000) {

      int soundValue = analogRead(SOUND_PIN);
```

```

    int change = abs(soundValue - baseline);

    if (change > maxChange) {
        maxChange = change;
    }

    delay(10);
}

// SHOW RESULT
lcd.clear();

if (maxChange >= THRESHOLD) {
    lcd.setCursor(0, 0);
    lcd.print("CRUNCH DETECTED");
    lcd.setCursor(0, 1);
    lcd.print("Score: ");
    lcd.print(maxChange);
}
else {
    lcd.setCursor(0, 0);
    lcd.print("NO CRUNCH");
    lcd.setCursor(0, 1);
    lcd.print("Score: ");
    lcd.print(maxChange);
}

delay(3000);

// READY FOR NEXT TEST
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("PAPPADAM TEST");
lcd.setCursor(0, 1);
lcd.print("Press START");

// Wait until button is released
while (digitalRead(BUTTON_PIN) == LOW) {
    delay(10);
}
}
}

```

</code></pre>

<h2>6. Project Team & Responsibilities</h2>

Sethuparvathy K J (Team Lead): Concept development, embedded system design, state-machine algorithm implementation in C++, and acoustic threshold calibration.

Shaheem: Hardware circuit assembly, component layout optimization, and physical testing execution.

</body>

</html>